Deep Learning- Final Commission Project Report

Milestone 0: Form Your 5-Member Groups

Ananya Prabhu - Project Manager
Rimmi Bhadani - Communications Manager
Jiao Chen - Final Performance Coordinator
Inoka Ranatunge - Final Performance Coordinator
Methruchi Peiris - Final Performance Coordinator.

Milestone 1: The Groundwork - Data Preparation

A total of 220 wineglass images were collected, covering a wide range of visual conditions to ensure model robustness.

- The dataset consisted of:
  - 100 self-captured images taken with a mobile phone under different lighting conditions and backgrounds and 100 images gathered from online sources, including product photos, lifestyle images, and open datasets.
  - Also, we collected 20 negative images, containing scenes without wine glasses (e.g., cups, bottles, plates)

Since the raw images came from mixed sources, their sizes varied significantly. To ensure consistent training performance and avoid excessively large files, all images were processed to a uniform resolution range of 300–500 KB.
This step helped to keep file sizes manageable for YOLO workflows.

Milestone 2: Core Challenge - CNN Optimization and Diagnosis

With the collected data, two zip files were generated containing images of 'wine glasses' and 'not a wine glass'.
After extracting the two zip files via the operating system, the data was organised into three directories – train, validation, and test – each containing two subdirectories: wine_glass and not_a_wine_glass.
Initially, we used a split ratio of 75%, 15%, and 15%. Since the validation accuracy did not improve, we tried 75%, 20%, and 5%. Finally, we proceeded with 80%, 15%, and 5%, understanding that the model required more training data. However, the results were not promising, even after applying several callbacks, such as early stopping and reduced learning rate. In addition to callbacks, we applied L2 regularisation and dropout, but the results remained unsatisfactory.
As the next step, we changed the dataset to include clearer images. We then split the dataset again using the same ratio of 80%, 15%, and 5%.
After normalising the train and validation data, the model was trained with the following architecture:

Component	Hyperparameter / Setting
Input	(128, 128, 3)
Conv Layer 1	16 filters, 3×3 kernel, ReLU activation
MaxPooling Layer 1	2×2 pool size
Conv Layer 2	32 filters, 3×3 kernel, ReLU activation
MaxPooling Layer 2	2×2 pool size
Conv Layer 3	64 filters, 3×3 kernel, ReLU activation
MaxPooling Layer 3	2×2 pool size
Conv Layer 4	128 filters, 3×3 kernel, ReLU activation
MaxPooling Layer 4	2×2 pool size
Flatten	Converts 2D feature maps to 1D vector
Dense Layer 1	128 units, ReLU activation
Dense Layer 2 (Output)	1 unit, Sigmoid activation (binary)
Optimizer	Adam
Loss Function	Binary crossentropy
Metrics	Accuracy
Epochs	10

The following table presents the training and validation results with the initial architecture, the steps taken to improve the results, the improved hyperparameters, and the corresponding observations.

Improved hyperparameters	**Training and validation results**	**Observations**
With the initial architecture	Training Accuracy 90% Validation Accuracy -90%	High bias of 10% and no human error
Increased the number of epochs to 50 and increased to 4 layers to make the model complex	Training Accuracy: 100.00% Validation Accuracy: 93.33%	A variance of 6,67%
Introduced early stopping with monitor='val_loss', and patience=8	Epoch 22: early stopping Restoring model weights from the end of the best epoch: 14. Training Accuracy: 90.00% Validation Accuracy: 90.00%	Both training and validation declined. Indicating a bias of 10% with no variance
• Introduced L2 regularization to the 1st layer and dropout of 0,5% for the final layer (This was after experimenting with many ways to improve accuracy) • With 100 epochs • Added a smaller learning rate of 1e-4	Epoch 81: early stopping Restoring model weights from the end of the best epoch: 73. Training Accuracy: 99.37% Validation Accuracy: 90.00%	No bias yet high variance. However, we could not increase the validation accuracy after this point

We calculated the evaluation metrics with the test data, and the results were as follows,
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
Finally, we saved the model weights to wineglass_classifier.h5

Milestone 3: Transfer Learning for Object Detection

With the collected dataset, a total of 220 images were prepared, including samples containing wine glasses and images without wine glasses as negative examples. After exporting the annotated data from Roboflow, the dataset followed the YOLOv5 PyTorch structure, organised into three directories: train, valid, and test, each containing separate images and labels folders.
Roboflow initially applied its default split, producing:
Train: 154 images, Validation: 44 images, Test: 22 images
Since this split already matched the YOLOv5 requirements and the dataset size was limited, we kept the proportions and proceeded directly with training.

Model Training
A pre-trained YOLOv5s model (yolov5s.pt) was used for fine-tuning. Transfer learning was chosen due to the relatively small dataset size, allowing the model to leverage pre-learned features.
Training command:
python train.py --img 640 --batch 16 --epochs 50 --data data.yaml --weights yolov5s.pt
During training, the model converged well, achieving high performance on both the training and validation sets. The loss curves indicated stable learning without signs of overfitting. The model consistently detected wine glasses across different lighting and background conditions.
The training completed successfully in 1.312 hours. After training, YOLOv5 produced two model weight files (last.pt and best.pt). The validation phase showed consistent and strong performance.
A summary of the final performance metrics is shown below:

Metric	Value
Precision(P)	0.955
Recall(R)	0.844
mAP50	0.919
Number of classes	1
Validation images	44
Total parameters	7012822

These results indicate that the model learned the wineglass class effectively, achieving high precision and strong overall detection performance.
Detection command:
python detect.py --weights runs/train/exp/weights/best.pt --source test/images
YOLOv5 generated output images with bounding boxes and also text files containing normalized bounding-box coordinates. The detection results demonstrated high confidence scores (0.90+) and precise bounding-box placement.
Final Output:
The trained model weights were saved as:
best.pt — YOLOv5 fine-tuned model
wineglass_classifier.h5 — exported weights for later integration

Milestone 4: Creative Enhancement - YOLO → GAN

We integrated the YOLOv5 object detection output with pretrained neural style transfer model. We used pre-trained neural style-transfer network, which acts like a generator network but does not require adversarial training. The pipeline is efficient and compatible because of pretrained Fast Neural Style Transfer models like mosaic, candy, rain princess and udnie. These models are feed-forward transformer networks that take an image patch as input and output a styled version in single forward pass.
We integrated YOLOv5 object detection output with models to enhance detected wine glass. YOLO model from milestone 3 produced bounding box label text files. The format of these files was class, x_center, y_center, width, height. The values were normalized, so first step was to convert them back to pixel coordinates to precisely crop the wine glass from test images.
Further, the image was resized while maintaining the aspect ratio and adjusted so that height and with were divisible by 4, which was required for pretrained TransformerNet models. We used multiple artistic models. The models were loaded by modifying state dictionary to remove deprecated parameters from older PyTorch version.
Once the patch process for the wine glass was done, the styled output was resized back to match the original YOLO bounding box dimensions. A Gaussian blurred soft mask was used when pasting the stylized patch to make sure smooth reintegration into original image. This prevented harsh edges and allowed styled object to blend naturally into its surroundings.

Output of Milestone 4

Original Image

Mosaic Model

Candy Model

Udnie Model

Rain-Princess Model

Challenges Faced

1. Find compatible style transfer models: Most of the pretrained models were incompatible with current PyTorch version.
2. Model loading issues: Older TransformerNet models included obsolete batch-normalization keys that caused errors during loading. These were removed manually before inference.
3. Handling path resizing: The models needed dimensions divisible by four. YOLO-cropped patches didn't meet this need, so careful resizing was done to preserve object shape and visual quality.

Conclusion

This project built a complete deep-learning pipeline capable of detecting wine glasses and creatively enhancing them using neural style transfer. We began with raw data collection and preprocessing. Telge and Inoka developed and evaluated the custom CNN classifier, which highlighted the limitations of a small and varied dataset. Chen then fine-tuned the YOLOv5 detection model, achieving strong and reliable performance and demonstrating the strengths of transfer learning for object-recognition tasks. The final integration of style-transfer models, handled by Rimmi and Ananya, required careful management of model compatibility issues, bounding-box transformations, and patch-level processing, ultimately producing smooth and visually coherent stylized outputs.

Overall, the project provided hands-on experience across multiple areas of modern computer vision, including dataset construction, neural-network training, performance diagnosis, object detection, and generative image manipulation. The final system combines technical accuracy with creative expression, showing how analytical and artistic deep-learning methods can complement each other. This assignment offered insight into real-world model development and emphasized the importance of iteration, experimentation, and teamwork in achieving a successful end-to-end solution.